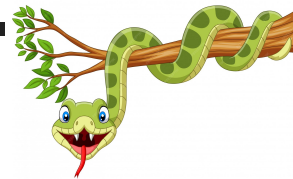
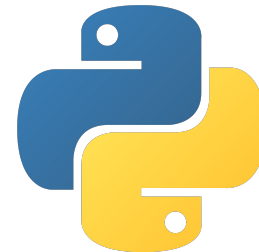




ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR

Membre de
HONORIS UNITED UNIVERSITIES

Programmation Python 3



Plan du cours

PARITE 1 : Introduction à Python

- Introduction à Python : Installation, IDEs, et premiers scripts, types de données (int, float, str, bool , complex)
- Structures de contrôle : if, else, elif, boucles for, while
- Conteneurs standard : listes, tuples, dictionnaires, ensembles
- Fonctions et méthodes de base

PARITE 2 : Programmation Orientée Objet en Python

- Classes et objets, attributs et méthodes spéciales (`__init__`, `__str__`, etc.)
- Héritage et polymorphisme, Encapsulation et abstraction
- Gestion des erreurs et exceptions (try, except)

PARITE 3 : Bibliothèques Python essentielles



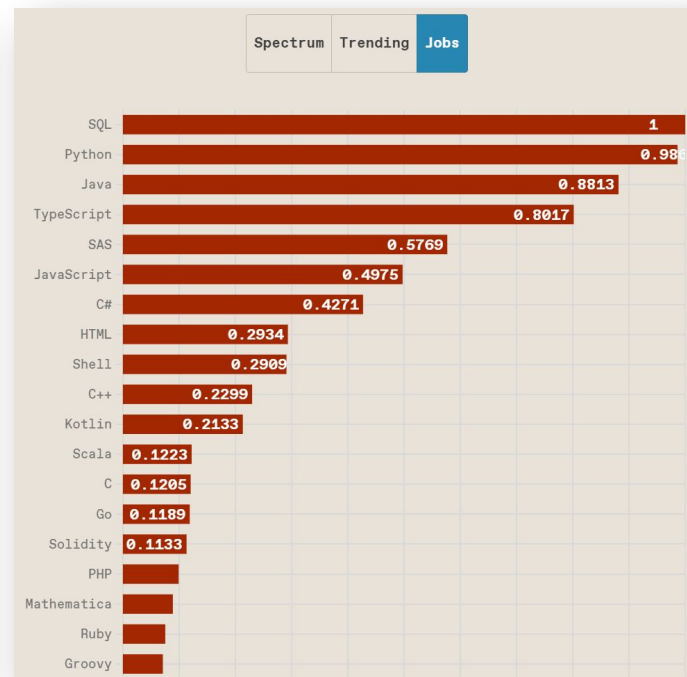
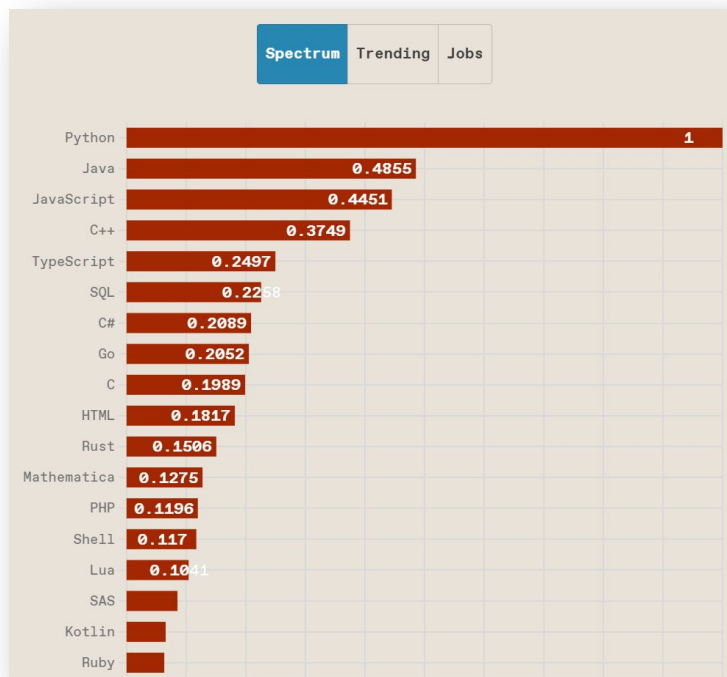
- Chapitre 1 -

Introduction à Python3 et types de données

Mais pourquoi donc apprendre Python ?

- Python est un langage facile à apprendre, son code est réputé clair et concis.
- C'est un langage à usage général, multi-plateforme et open source. Il est accompagné d'une importante bibliothèque standard.
- Python dispose de PyPI (Python Package Index : <https://pypi.org>), un dépôt en ligne de dizaines de milliers de packages qui offrent des solutions pour résoudre des problèmes dans des domaines divers.
- Python est apprécié pour sa simplicité, sa lisibilité et sa polyvalence, ce qui en fait un choix populaire pour les débutants et

Classement des Meilleurs langages en 2024



Reference : [*IEEE Spectrum's Top Programming Languages 2024 rankings*](#)

Survol historique

- 1991 : Guido VAN ROSSUM travaille aux Pays-Bas sur le projet AMOEBA, un système d'exploitation distribué. Il conçoit Python à partir du langage ABC et publie la version 0.9.0 sur un forum Usenet,
- 1996 : sortie de Numerical Python, ancêtre de numpy,
- 1999 : premières journées Python-France à l'Onera ²,
- 2001 : naissance de la PSF (Python Software Foundation), les versions se succèdent... Un grand choix de modules est disponible, des colloques annuels sont organisés, Python est enseigné dans plusieurs universités et est utilisé en entreprise...,
- 2006 : première sortie de IPython,
- fin 2008 : sortie simultanée de Python 2.6 et de Python 3.0

Techniques de production des programmes

Chaque microprocesseur possède un langage propre, directement exécutable : le langage *machine binaire*, c'est le *seul* que l'ordinateur puisse utiliser. Ce langage n'est pas portable.

Les *langages de haut niveau* (par exemple C, Java, PHP, Python...) sont généralement normalisés. Ils permettent le portage d'une machine à l'autre. Ils sont traduits dans le langage machine adapté au microprocesseur par un *compilateur* ou un *interpréteur*.

Compilation vs Interprétation

La *compilation* est la traduction du texte du programme (dit *source*) en une représentation quasi prête pour le microprocesseur (dit *objet*). Elle comprend au moins quatre phases (trois phases d'analyse – lexicale, syntaxique et sémantique et une phase de production de code *objet*).



FIGURE 1.2 – Chaîne de compilation

Compilation vs Interprétation

Dans la technique de l'*interprétation*, chaque ligne de la *source* analysée est traduite au fur et à mesure en instructions directement exécutées. Aucun programme objet n'est généré. Cette technique est très souple, mais les codes générés sont peu performants : l'interprétation doit être réalisée à chaque nouvelle exécution...



FIGURE 1.3 – Chaîne d'interprétation

Technique de production de Python

Le concepteur de Python a opté pour une technique mixte : l'*interprétation du bytecode compilé*.

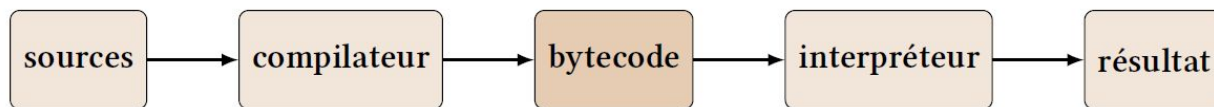


FIGURE 1.4 – Interprétation du bytecode compilé

Pour **exécuter un programme**, Python charge le fichier source (**.py**) en mémoire vive, en fait l'analyse, produit le bytecode et enfin l'exécute. Afin de ne pas refaire inutilement les phases d'analyse et de production, Python sauvegarde le bytecode produit (dans un fichier **.pyo**) et recharge simplement le fichier bytecode s'il est plus récent que le fichier source dont il est issu.

Construction des programmes

Le génie logiciel étudie les méthodes de construction des programmes. Plusieurs modèles sont envisageables, entre autres :

- la **méthodologie procédurale**. On s'efforce de décomposer un problème complexe en sous-programmes plus simples.
- la **méthodologie objet**. Centrée sur les données, elle est considérée plus stable dans le temps et meilleure dans sa conception.

Python offre les *deux* techniques (on parle de *paradigmes*), que l'on peut mélanger suivant les besoins et suivant ce qui apparaît comme le plus adapté au problème à résoudre.

Modes d'exécution d'un code Python

- Le mode le plus direct et intuitif d'exécution de Python est l'utilisation interactive de l'*interpréteur*.
- Lorsqu'on tape la commande **python3** dans une console (ou une invite de commande), une « invite » apparaît.

```
>>> 5 + 3   Python affiche l'invite. L'utilisateur tape une expression.  
8           Python évalue et affiche le résultat...  
>>>        ... puis réaffiche l'invite.
```

Modes d'exécution d'un code Python

Mais dès que l'on travaille avec plus que quelques lignes de code, le mode interprété devient malcommode. On passe alors en *mode script* : on enregistre un ensemble d'instructions Python dans un fichier source (ou *script*) grâce à un éditeur.

Ce script est exécuté ultérieurement (et autant de fois que l'on veut) par une commande ou par une touche du menu de l'éditeur, il peut être corrigé puis réexécuté dans son ensemble.

Identificateurs

Comme tout langage, Python utilise des identificateurs de *variables* pour nommer tout ce qui est manipulé.

Un **identificateur de variable** Python est une suite non vide de caractères, de longueur quelconque, formée d'**un** *caractère de début* (n'importe quelle lettre Unicode ou le caractère souligné) et de **zéro** (au sens d'« aucun ») **ou plusieurs** *caractères de continuation* (lettre Unicode, caractère souligné ou chiffre).

D'une part, les identificateurs de variables sont sensibles à la casse (distinction minuscule/majuscule) et, d'autre part, ils ne doivent pas faire partie des mots réservés de Python 3

Mots-clés de Python 3

<code>False</code>	<code>await</code>	<code>else</code>	<code>import</code>	<code>pass</code>
<code>None</code>	<code>break</code>	<code>except</code>	<code>in</code>	<code>raise</code>
<code>True</code>	<code>class</code>	<code>finally</code>	<code>is</code>	<code>return</code>
<code>and</code>	<code>continue</code>	<code>for</code>	<code>lambda</code>	<code>try</code>
<code>as</code>	<code>def</code>	<code>from</code>	<code>nonlocal</code>	<code>while</code>
<code>assert</code>	<code>del</code>	<code>global</code>	<code>not</code>	<code>with</code>
<code>async</code>	<code>elif</code>	<code>if</code>	<code>or</code>	<code>yield</code>

TABLEAU 2.1 – Python 3.8 compte 35 mots-clés

Commentaires, une affaire de style

PEP 8 (pour Python Extension Proposal) est un ensemble de règles qui permet d'homogénéiser le code et d'appliquer de bonnes pratiques.

La signification de parties de votre code doit être expliquée par un commentaire. En Python, un commentaire commence par le caractère `#` et s'étend jusqu'à la fin de la ligne. La recommandation PEP 8 est faite sur l'ajout d'espacements minimums pour augmenter la lisibilité (matérialisés par des points) :

```
#.Commentaire_bloc  
#.sur  
#.plusieurs lignes  
  
variable.=.fonction(arg)..#.Commentaire_ligne
```

Commentaires, une affaire de style

Les docstrings sont entourées de triples guillemets (""") et peuvent s'étendre sur plusieurs lignes. Elles sont très utiles pour expliquer le but et le fonctionnement de votre code de manière détaillée.

```
def ma_fonction():  
    """  
    Cette fonction ne fait rien de particulier.  
  
    Elle est utilisée uniquement pour démontrer comment écrire  
    un commentaire sur plusieurs lignes en Python.  
  
    Les docstrings sont souvent utilisées pour documenter les  
    fonctions,  
    les classes et les modules.  
    """  
    pass
```

Nommage des variables

Il est important d'utiliser une politique cohérente de nommage des variables. Le style utilisé dans ce présent cours respecte au mieux la recommandation de la communauté Python :

Exemples :

```
NB_ITEMS = 12                # Appelé "UPPER_CASE_WITH_UNDERSCORES"  
class MaClasse: pass         # Appelé "CamelCase" ou "CapitalizedWords"  
monmodule2                   # Appelé "lower_case"  
def ma_fonction(): pass      # Appelé "lower_case_with_underscores"  
mon_id = 5                   # idem
```

Notion d'expression

Une **expression** est une portion de code que l'interpréteur Python peut **évaluer** pour obtenir une **valeur**. Les expressions peuvent être **simples** ou **complexes**. Elles sont formées d'une combinaison de littéraux (représentant directement des valeurs), d'identificateurs et d'opérateurs.

Par exemple :

```
15.3  
4 + 3 * sin(pi)  
"- "*10 + "Titre" + "- "*10
```

Variable, identifiant et objet

La notion d'*objet* est importante car toutes les données d'un programme Python sont représentées par des objets.

Un **objet** peut être caractérisé comme une **donnée** définie par un morceau de code, possédant un *identifiant*, un *type* et une *valeur*.

Prenons l'exemple d'un objet chaîne de caractères. En Python une telle chaîne s'écrit entre apostrophes :

```
>>> 'Alain'
```

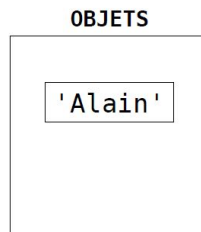
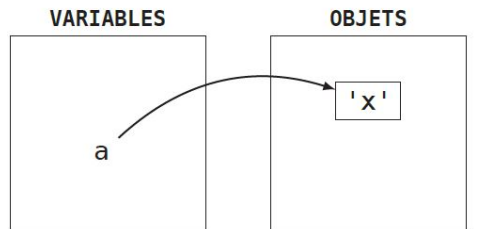


FIGURE 2.1 – Un objet dans l'espace des objets

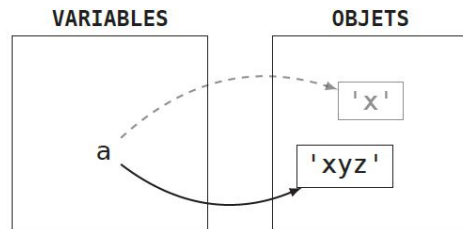
Variable et objet : id d'objet

```
>>> a = 'x'
>>> id(a)
140224486267736
>>> a += 'yz'
>>> a
'xyz'
>>> id(a)      # L'identifiant a changé (objet immutable)
140224469952584
```



(a) Affectation d'une chaîne de caractères

```
>>> a = 'x'
```



(b) Affectation augmentée de la chaîne

```
>>> a += 'yz'
```

Variable et objet

L'**identifiant** d'un objet ne change jamais après sa création. Il **représente de façon unique l'objet** au cours de sa vie en mémoire. Le **type de l'objet** détermine les opérations (les *méthodes*) que l'on peut lui appliquer et définit aussi les valeurs possibles (les *données*) pour les objets de ce type.

Exemple, les objets de type **str** possèdent de nombreuses méthodes, par exemple la méthode **upper** qui met la chaîne en majuscule :

```
>>> type('Alain')  
<class 'str'>
```

```
>>> 'Alain'.upper()  
'ALAIN'
```

Affectation

En Python, on dit que « les variables référencent les objets ». Pour lier une variable à un objet, on utilise la notation d'affectation.

Syntaxe



Notation d'affectation :

```
>>> prenom = 'Alain'  
>>> prenom.upper() # On manipule l'objet 'Alain' au moyen de sa référence prenom  
'ALAIN'
```

Attention



Même si on utilise le signe `=`, l'affectation *n'a rien à voir* avec l'égalité en maths !

Affectation

Lors d'une affectation, l'interpréteur Python exécute trois opérations représentées :

1. création de l'objet **'Alain'** dans l'espace des objets
2. création de la variable « **prenom** » dans l'espace des variables
3. création d'une référence entre la variable et l'objet

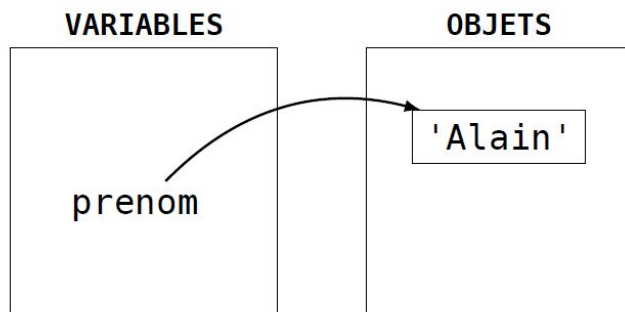


FIGURE 2.2 – Opération d'affectation : la variable `prenom` référence l'objet **'Alain'**

Réaffectation

Si maintenant on affecte la variable **prenom** à la chaîne '**Bob**', que va-il se passer ?

Python crée l'objet '**Bob**' dans l'espace des objets puis, comme la variable **prenom** existe déjà dans l'espace des variables, Python la *déréférence* et lui fait maintenant référencer l'objet '**Bob**'.

```
>>> prenom = 'Alain'
>>> type(prenom)
<class 'str'>
>>> prenom = 'Bob' # Réaffectation (même type)
>>> type(prenom)
<class 'str'>
```

Quand un objet n'est plus référencé, un mécanisme automatique de gestion de la mémoire entre en jeu : le **garbage collector** de Python va libérer la mémoire occupée par cet objet.

Typage dynamique

Et si maintenant la nouvelle affectation de **prenom** est un objet d'un autre type, par exemple l'objet **7** de type entier, quel va être le type de **prenom** ?

Comme précédemment, Python dérèfère **prenom** et lui fait référencer l'objet 7

```
>>> prenom = 7          # Réaffectation (autre type) : typage dynamique
>>> type(prenom)
<class 'int'>
```

Réaffectation et typage dynamique

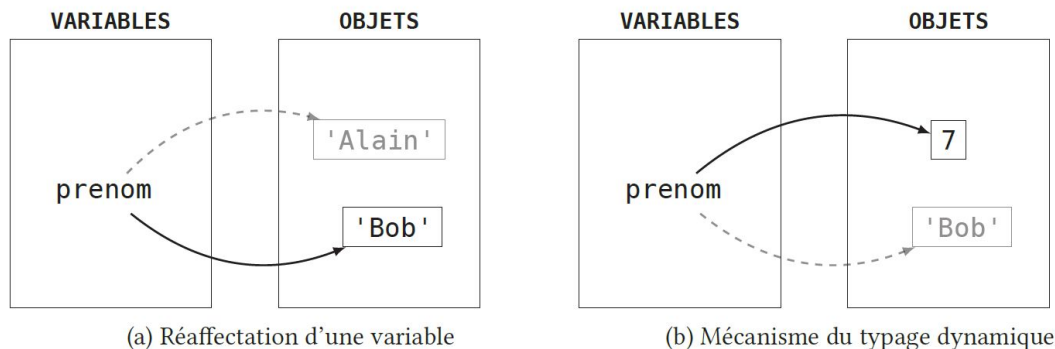


FIGURE 2.3 – Réaffectation et typage dynamique

Python est un langage à *typage fort*, ce qui signifie qu'un objet créé va garder son type pendant toute son existence alors qu'une variable peut référencer des objets de types différents durant l'exécution du programme. C'est précisément ce mécanisme que l'on nomme le *typage dynamique*.

Attention

!! En Python, le type n'est pas lié à la variable qui référence l'objet mais à l'objet lui-même.

Variantes de l'affectation

Outre l'affectation simple, on peut aussi utiliser les formes suivantes :

```
>>> v = 4                # Affectation simple
>>> v += 2               # Affectation augmentée. Idem à v = v + 2 si v est déjà référencé

>>> v
6
>>> c = d = 8            # d reçoit 8, puis c reçoit d. Ils référencent la même donnée (alias)
>>> c, d                 # Demande des deux valeurs (sous la forme d'un tuple)
(8, 8)
>>> e, f = 2.7, 5.1      # Affectation parallèle par décapsulation d'un tuple
>>> e, f
(2.7, 5.1)
>>> a, b = 3, 5
```

Variantes de l'affectation

Dans une affectation, le membre de gauche pointe sur le membre de droite, ce qui nécessite d'évaluer la valeur du membre de droite avant de référencer le membre de gauche. On voit dans l'affectation parallèle l'importance de cette séquence temporelle.

```
>>> a, b = a + b, a * 2 # Toutes les expressions sont évaluées avant la première affectation
>>> a, b
(8, 6)
```

Expression d'affectation « := »

À partir de Python 3.8, il est possible d'effectuer une affectation au sein d'une expression en utilisant l'opérateur `:=` (appelé *morse*). Ceci permet entre autres d'éviter de répéter des expressions dans le cadre de boucles conditionnelles.

Par exemple :

```
>>> while (s := input("Texte: ")) != "":  
    print(f"Majuscules: {s.upper()}")
```

```
Texte: Bonjour  
Majuscules: BONJOUR  
Texte: Ça va ?  
Majuscules: ÇA VA ?  
Texte:
```

Suppression d'une variable

Puisque tout est dynamique en Python, il est possible, au cours de l'exécution, de supprimer une variable, donc de supprimer un nom qui référence une donnée.

Python fournit pour cela l'instruction **del** :

```
>>> a = 3
>>> del a
>>> a      # La variable vient d'être supprimée : erreur
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'a' is not defined
```

Comment trouver une documentation

Python propose plusieurs façons de se documenter.

Dans tout interpréteur, on peut utiliser la fonction built-in `help(objet)` à condition, bien sûr, que l'objet soit présent dans l'espace de travail, sinon il faut l'importer :

```
>>> help(len)
Help on built-in function len in module builtins:

len(obj, /)
    Return the number of items in a container.
```

Exercice

Tester la fonction `id()` en Python

- Créez une variable `a` et assignez-lui la valeur 5.
- Créez une seconde variable `b` et affectez-lui la valeur de `a` (c'est-à-dire `b = a`).
- Affichez le résultat de la comparaison entre les identifiants de `a` et `b` en utilisant la fonction `id()` (la comparaison doit vérifier si les identifiants de `a` et `b` sont identiques).

Types de données entiers (int)

Python offre deux types entiers standard : **int** et **bool**.

Le type **int** est la représentation informatique de l'ensemble des entiers naturels mathématiques. Il n'est limité en taille que par la mémoire de la machine. Les entiers sont représentés en décimal par défaut, mais on peut aussi utiliser les bases suivantes :

```
>>> 2013          # Décimal (base 10) par défaut
2013
>>> 0b11111011101 # Binaire (base 2) avec le préfixe 0b
2013
>>> 0o3735        # Octal (base 8) avec le préfixe 0o
2013
```

```
>>> 0x7dd         # Hexadecimal (base 16) avec le préfixe 0x
2013
>>> # Représentations binaire, octale et hexadécimale de l'entier 179
>>> bin(179), oct(179), hex(179)
('0b10110011', '0o263', '0xb3')
```

Ces dernières opérations correspondent à des changements de bases classiques

Opérations arithmétiques

Les principales opérations :

Bien remarquer le rôle des deux opérateurs de division :

/ : produit une division flottante, même entre deux entiers ;

// : produit une division entière.

```
>>> 20 + 3
23
>>> 20 - 3
17
>>> 20 * 3
60
>>> 20 ** 3
8000
>>> 20 / 3          # Division flottante
6.666666666666667
>>> 20 // 3         # Division entière
6
>>> 20 % 3          # Modulo (reste de la division entière)
2
>>> divmod(20, 3)   # Division entière et modulo (reste)
(6, 2)
>>> abs(3 - 20)     # Valeur absolue
17
```

Types de données entiers (bool)

Principales caractéristiques du type **bool** :

- Deux valeurs possibles : **False** (faux), **True** (vrai).
- Conversion automatique (ou « transtypage ») des valeurs des autres types vers le type booléen.
- Opérateurs de comparaison entre deux valeurs comparables, produisant un résultat de type **bool** : `==` (égalité), `!=` (différence), `>` , `>=` , `<` et

```
>>> 2 > 8
```

```
False
```

```
>>> 2 <= 8 < 15
```

```
True
```

Types de données entiers (bool)

- Opérateurs logiques : **not**, **or** et **and**. En observant les tables de vérité des opérateurs **and** et **or**, on remarque que :
 - dès qu'un premier membre possède la valeur **False**, l'expression **False and** expression2 vaudra **False**. On n'a donc pas besoin d'évaluer expression2 ;
 - de même, dès qu'un premier membre a la valeur **True**, l'expression **True or** expression2 vaudra **True**, quelle que soit la valeur de expression2.

Cette
court

```
>>> (3 == 3) or (9 > 24)
True
>>> (9 > 24) and (3 == 3)
False
```

incipe du *shortcut* » ou évaluation « **au plus** » de l'évaluation des expressions booléennes :

Types de données entiers (bool)

Pour être sûr d'avoir un résultat booléen avec une expression reposant sur des valeurs *transtypées*, appliquez **bool()** sur l'expression. Python effectue des conversions de type automatique sur les données. Mais le résultat de l'évaluation utilise les valeurs d'origine :

```
>>> 'a' or False # 'a' est évalué à vrai
'a'
>>> 0 or 56      # 0 et 56 sont transtypés en booléen.
56               0 vaut False et les autres valeurs True
>>> b = 0
>>> b and 3>2    # b est transtypé en booléen
0
```

Types de données flottants

Un **float** est noté avec un point décimal (jamais avec une virgule) ou, en notation exponentielle, avec un **e** ou un **E** symbolisant le « 10 puissance » suivi des chiffres de l'exposant.

Par exemple : 2.718, .02, 3E10, -1.6e-19, 6.023E23.

- Les flottants supportent les mêmes opérations que les entiers.
- Ils ont une précision limitée

L'import du module standard `math` autorise toutes les opérations mathématiques usuelles. Par exemple :

```
>>> import math
>>> math.sin(math.pi/4)
0.7071067811865475
>>>> math.degrees(math.pi)
180.0
>>>> math.hypot(3.0, 4.0)
5.0
>>> math.log(1024, 2)
10.0
```

Type complex

Les complexes sont écrits en notation cartésienne formée de deux flottants. La partie imaginaire est suffixée par **j**

```
>>> 1j
1j
>>> (2+3j) + (4-7j)
(6-4j)
>>> (9+5j).real
9.0
>>> (9+5j).imag
5.0
>>> (abs(3+4j)) # Module d'un complexe
5.0
```

Type complex

Un module mathématique spécifique (**cmath**) leur est réservé, exemple :

```
>>> import cmath
>>> cmath.phase(-1+0j)
3.141592653589793
>>> cmath.polar(3+4j)
(5.0, 0.9272952180016122)
>>> cmath.rect(1., cmath.pi/4)
(0.7071067811865476+0.7071067811865475j)
>>> cmath.sqrt(-5)
2.23606797749979j
```

Exercice

- Ecrire un programme qui affiche la résistance équivalente à trois résistances R1, R2, R3 (type **int**) :
 - $R_{\text{sér}} = R1 + R2 + R3$
 - $R_{\text{par}} = (R1 * R2 * R3) / (R1 * R2 + R1 * R3 + R2 * R3)$

Chaînes de caractères

En Python, les chaînes de caractères représentent une séquence de caractères Unicode. Leur type de données, noté **str**, est **immutable**, ce qui signifie qu'un objet, une fois créé en mémoire, ne pourra plus être changé ; toute transformation aboutira à la création d'un *nouvel* objet distinct.

L'utilisation de l'apostrophe (**'**) à la place du guillemet (**"**) autorise l'inclusion d'une notation dans l'autre. La notation entre trois guillemets (**"""**) permet de composer des chaînes sur plusieurs lignes contenant elles-mêmes des guillemets simples ou doubles.

Exemple : Chaînes de caractères

```
>>> guillemets = "L'eau vive"
>>> apostrophes = 'Il a écrit : "Ni le mort ni le vif, mais le merveilleux !"'
>>> doc = """
forme multiligne très utile pour la documentation d'un script, fonction ou classe,
ou pour inclure un fragment de programme dans une chaîne de caractères :

for i in range(2, 2*n + 1):
    if i%2 == 0: # indice pair
        monge.insert(0, i)
    else:
        monge.append(i)
"""
```

Séquences d'échappement

À l'intérieur d'une chaîne, le caractère antislash (\) donne une signification spéciale à certaines séquences de caractères.

```
>>> "\N{pound sign} \u00A3 \U000000A3"  
£ £ £  
>>> "d \144 \x64"  
d d d  
>>> r"d \144 \x64" # La notation r"... " (raw) désactive la signification spéciale du caractère "\"  
d \144 \x64
```

Séquences d'échappement

Séquence	Signification
\	saut de ligne ignoré (placé en fin de ligne)
\\	antislash
\'	apostrophe
\"	guillemet
\a	sonnerie (<i>bip</i>)
\b	retour arrière
\f	saut de page
\n	saut de ligne
\r	retour en début de ligne
\t	tabulation horizontale
\v	tabulation verticale
\N{nom}	caractère sous forme de code Unicode nommé
\uhhhh	caractère sous forme de code Unicode 16 bits sur 4 chiffres hexadécimaux
\Uhhhhhhh	caractère sous forme de code Unicode 32 bits sur 8 chiffres hexadécimaux
\ooo	caractère sous forme de code octal sur 3 chiffres octaux
\xhh	caractère sous forme de code hexadécimal sur 2 chiffres hexadécimaux

TABLEAU 2.2 – Séquences d'échappement des chaînes de caractères

Opérations

Outre les fonctions et méthodes que nous allons voir, les quatre opérations suivantes : longueur, concaténation, répétition et test d'appartenance s'appliquent au type **str** :

```
>>> len("abcde")           # Longueur (nombre de caractères dans la chaîne)
5
>>> "abc" + "defg"         # Concaténation (mise bout à bout de deux chaînes ou plus)
'abcdefg'
>>> "Fi! " * 3              # Répétition (avec * entre une chaîne et un entier)
'Fi! Fi! Fi! '
>>> 'thon' in 'Python!'     # L'opérateur 'in' teste l'appartenance d'un élément à une chaîne
True
```

Fonctions vs méthodes

On peut agir sur une chaîne en utilisant des *fonctions* (notion procédurale) communes à tous les types séquences ou conteneurs, ou bien des *méthodes* (notion objet) spécifiques aux chaînes :

```
>>> len('Les auteurs')      # Syntaxe de l'appel d'une fonction
11
>>> "abracadabra".upper()   # Syntaxe de l'appel d'une méthode (notation pointée)
"ABRACADABRA"
```

Méthodes de test de l'état d'une chaîne

Il s'agit de méthodes à valeur booléenne, c'est-à-dire qu'elles retournent la valeur **True** ou **False**. Voici quelques exemples de ces méthodes de test :

```
>>> 'Le petit PRINCE'.isupper()           # Tout est en majuscule
False
>>> 'Le Petit Prince'.istitle()           # Chaque mot commence par une majuscule
True
>>> 'Prince'.isalpha()                    # Ne contient que des caractères alphabétiques
True
>>> 'Un'.isdigit()                        # Ne contient que des caractères numériques
False
>>> 'Le Petit Prince'.startswith('Le ')   # Commence par...
True
>>> 'Le Petit Prince'.endswith('prince')  # ... finit par (différence de casse sur le P)
False
```

Méthodes retournant une nouvelle chaîne

Comme les chaînes sont immutables (c'est-à-dire que leur contenu ne peut pas changer), les méthodes qui effectuent des modifications retournent de *nouvelles* chaînes. En voici quelques exemples :

```
>>> 'Le petit PRINCE'.lower()           # Tout en minuscule
'le petit prince'
>>> 'Le petit PRINCE'.upper()           # Tout en majuscule
'LE PETIT PRINCE'
>>> 'Le petit PRINCE'.swapcase()         # Inverser la casse
'LE PETIT prince'
>>> 'Le petit PRINCE'.center(31, '~')    # Chaîne centrée
'~~~~~Le petit PRINCE~~~~~'
>>> 'Le petit PRINCE'.rjust(31, '^')      # Chaîne justifiée à droite
'^^^^^^^^^^^^^^^^Le petit PRINCE'
>>> ' Le petit Prince '.rstrip('e L')    # Suppression des 'e', espaces ou 'L' en début de chaîne
'petit Prince '
>>> 'Le petit Prince'.replace('petit', 'grand')
'Le grand Prince'
>>> 'Le petit Prince'.split()             # Découpe la chaîne suivant le séparateur
                                         # (séquence d'espaces par défaut)
['Le', 'petit', 'Prince']
```

Méthode retournant un index

find(sub[, start[, stop]]) : renvoie l'index de la chaîne sub dans la sous-chaîne start à stop, sinon renvoie -1.

rfind() effectue le même travail en commençant par la fin.

index() et **rindex()** font de même mais produisent une erreur (*exception*) si la chaîne sub n'est pas trouvée :

```
>>> 'Le petit Prince'.find('Pr')
9
>>> 'Le petit Prince'.index('bad')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: substring not found
```

Indexation simple

L'indexation utilise la notation `[index]` , dans lequel `index` est un entier signé (positif ou négatif) *qui commence à 0* et indique la position d'un caractère.

L'utilisation de valeurs d'index négatives permet d'accéder aux caractères par la fin de la chaîne :

```
>>> s = "Rayons X"    # len(s) ==> 8
>>> s[0]              # Premier caractère
'R'
>>> s[2]              # Troisième caractère
'y'
>>> s[-1]             # Dernier caractère
'X'
>>> s[-3]             # Antépénultième caractère
's'
```

`s = 'Rayons X'`

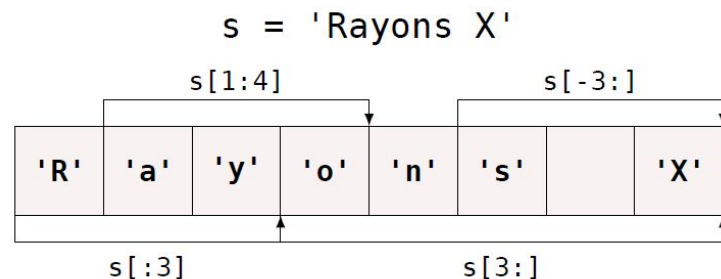
<code>s[0]</code>	<code>s[1]</code>	<code>s[2]</code>	<code>s[3]</code>	<code>s[4]</code>	<code>s[5]</code>	<code>s[6]</code>	<code>s[7]</code>
'R'	'a'	'y'	'o'	'n'	's'		'X'

`s[-8]` `s[-7]` `s[-6]` `s[-5]` `s[-4]` `s[-3]` `s[-2]` `s[-1]`

Indexation en tranches

L'opérateur de **slicing** ou « découpage » **[début:fin]** ou **[début:fin:pas]**, dans lequel **début** et **fin** sont des index de caractères et **pas** est un entier signé, permet d'extraire des tranches (ou sous-chaîne de caractères).

```
>>> s = "Rayons X"    # len(s) ==> 8
>>> s[1:4]            # De l'index 1 compris à 4 non compris
'ayo'
>>> s[-3:]            # De l'index -3 compris à la fin
's X'
>>> s[:3]             # Du début à l'index 3 non compris
'Ray'
>>> s[3:]             # De l'index 3 compris à la fin
'ons X'
>>> s[::2]            # Du début à la fin, de 2 en 2
'Ryn '
>>> s[::-1]           # Du début à la fin en pas inverse (retournement)
'X snoyaR'
```



Exercice

Imaginons que vous avez une chaîne de texte contenant des informations sur un employé et que vous devez extraire des informations spécifiques de cette chaîne.

`employe` = "Nom : John Doe | Âge : 30 | Département : IT | Emplacement : New York"

- Extraire le **nom** de l'employé, l'**âge** et le **département**.

Formatage de chaînes : f-strings

À partir de Python 3.6, le formatage de valeurs sous forme de chaînes a été intégré au langage avec la notation à base d'un préfixe **f** ou **F** devant une chaîne qui permet que celle-ci contienne des expressions évaluées lors de l'exécution.

Les expressions dans la chaîne sont simplement placées entre accolades **{...}** pour les identifier (pour éviter qu'une accolade ne soit considérée comme une délimitation d'expression, on la double).

```
>>> x = 4
>>> titre = "Python 3"
>>> f"J'ai commandé {x} exemplaires de {titre}."
"J'ai commandé 4 exemplaires de Python 3."
>>> f"Total: {{x={x}}}"
'Total: {x=4}'
```

Formatage de chaînes : f-strings

Les **expressions** peuvent utiliser tous les noms définis au moment où la chaîne est évaluée, ainsi que toute la richesse des expressions Python (variables, calculs, fonctions, méthodes, indexation...).

```
>>> f"J'ai commandé {x * 100} exemplaires de {titre.upper()}."
"J'ai commandé 400 exemplaires de PYTHON 3."
>>> f"'{titre}' fait {len(titre)} caractères."
"'Python 3' fait 8 caractères."
>>> f"Le début est {titre[:6]}."
"Le début est Python."
```

Formatage de chaînes : f-strings

Il faut toutefois faire attention à respecter les marqueurs de début/fin de chaîne en utilisant la marque alternative " ou ' au sein des expressions lorsqu'elles désignent des chaînes.

```
>>> f"Bonjour {'guido'.upper()}." # f-string entre ", chaîne interne entre  
'Bonjour GUIDO.'
```

Veiller également à mettre entre parenthèses les expressions qui pourraient contenir des accolades.

```
>>> f"Beaucoup d'exemplaires: {(True:'Vrai',False:'Faux')[x>1]}"  
"Beaucoup d'exemplaires: Vrai"
```

Formatage

Il est possible d'indiquer, juste après l'expression et le séparateur `:` des directives de formatage pour la valeur sous la forme :

`:`
**[drapeau][largeur][.précision][type
]**

Le drapeau permet de spécifier un **caractère de remplissage** (optionnel) et un **alignement** (`<` gauche, `^` centré, `>` droite) pour positionner le texte dans la largeur demandée.

Formatage

```
>>> f"Ils sont {x:12}"
'Ils sont           4'
>>> f"Le titre est {titre:12}"
'Le titre est Python 3'
>>> f"Le titre est {titre*3:12}"
'Le titre est Python 3Python 3Python 3'
>>> f"Le titre est {titre*3:^12.12}"
'Le titre est Python 3Pyth'
>>> f"Ils sont {x:>12}"
'Ils sont           4'
>>> f"Ils sont {x:<12}"
'Ils sont 4'
>>> f"Ils sont {x:^12}"
'Ils sont           4'
>>> f"Ils sont {x:-^12}"
'Ils sont -----4-----'
```

Par défaut entiers alignés à droite

Par défaut les chaînes sont alignées à gauche

Spécification de la largeur uniquement
→ la chaîne dépasse

Spécification largeur.précision
→ la chaîne est tronquée

Chaîne alignée à droite

Alignée à gauche

Centrée

Centrée avec un caractère de remplissage

Type de formatage : nombres entiers

Pour les **nombres entiers**, il existe quatre types de formatages : **d** (décimal par défaut), **x** (hexadécimal, **X** pour avoir les chiffres hexadécimaux en majuscule), **o** (octal) et **b** (binaire).

```
>>> n = 42
>>> f"d=>{n:d} x=>{n:x} o=>{n:o} b=>{n:b}"
'd=>42 x=>2a o=>52 b=>101010'
```

Type de formatage : nombres flottants

Pour les **nombres flottants**, il existe quatre types de formatages : **e** (ingénieur ou scientifique), **f** (formatage décimal par défaut, 6 chiffres après la virgule par défaut), **g** (adapté **e** ou **f** suivant l'ordre de grandeur pour obtenir une valeur lisible sur un espace raisonnable) et **%** (valeur $\times 100$ et place un symbole %).

Valeur	$\pi = 3.141592653589793$	$\pi \times 10^6$	$\pi \times 10^{-6}$
Formatage e	3.141593e+00	3.141593e+06	3.141593e-06
Formatage f	3.141593	3141592.653590	0.000003
Formatage g	3.14159	3.14159e+06	3.14159e-06

TABLEAU 2.3 – Formatage des nombres flottants

Type de formatage : nombres flottants

On peut utiliser là encore un drapeau **+** pour forcer la présence du signe, la largeur pour indiquer l'espace de formatage, et la précision pour indiquer le nombre de décimales.

```
>>> pi = 3.141592653589793
>>> f"{pi:+10.3f}"      # '      +3.142'
>>> f"{pi:0.30f}"      # '3.141592653589793115997963468544'
>>> f"{pi:0.2e}"       # '3.14e+00'
>>> f"{pi:0.3%}"       # '314.159%'
>>> f"{pi:0.2%}"       # '314.16%'
```

Type de formatage : Conversion textuelle

Hors spécifications de formatage particulières, la génération de la forme textuelle des valeurs en Python passe par deux fonctions, **str()** qui demande une forme orientée utilisateur, et **repr()** qui demande une représentation littérale (telle qu'on la trouve dans les sources des programmes).

Avec les *f-string*, il est possible d'utiliser les indications **!s** et **!r**, placées après les indications de formatage, pour demander qu'une de ces deux fonctions soit appliquée à la valeur à formater

```
>>> f"La chaîne du titre: {titre!r}"    # "La chaîne du titre: 'Python 3'"
>>> f"Le texte du titre: {titre!s}"    # 'Le texte du titre: Python 3'
```

Méthode .format()

La méthode .format() des chaînes utilise la même syntaxe que les *f-string* (mêmes écritures pour les options de formatage).

- Un nombre entier indique un argument positionnel fourni à **format** – par défaut, sans indication d'accès aux données, elles sont utilisées dans l'ordre où elles ont été fournies à **format**.
- Un nom indique un argument nommé fourni à **format**.

```
>>> "argument 0 {0:0>5}, argument 1 {1!r}, nom toto {toto}, nom auteur {gui}".format(4, "Python 3", gui="Guido", toto=3.14159)  
"argument 0 00004, argument 1 'Python 3', nom toto 3.14159, nom auteur Guido"
```

Opérateur % (le formatage historique)

C'est la plus ancienne façon en Python de formater des valeurs en chaînes de caractères, une adaptation de la syntaxe de la fonction `printf()` du langage C.

Au lieu d'indiquer l'expression et les options de formatage entre `{}`, on utilise dans la chaîne des caractères `%` qui signalent les endroits où il faut insérer des valeurs

```
>>> "%d %s %f" % (x, titre, pi)
'4 Python 3 3.141593'
```

```
>>> "%4d %20s %.2f" % (x, titre, pi)
'   4                Python 3 3.14'
```

Pour afficher effectivement un caractère `%`, il suffit de le doubler.

Entrées de base

Il s'agit de réaliser une *saisie* au clavier : la fonction standard `input()` interrompt le programme, affiche une éventuelle invite à l'écran et attend que l'utilisateur entre une donnée au clavier.

La fonction `input()` effectue toujours une saisie en *mode texte* (la valeur retournée est une chaîne) dont on peut ensuite changer le type (on dit aussi « transtyper ») :

```
>>> f = input("Entrez un flottant : ")
Entrez un flottant : 12.345
>>> type(f)
str
>>> g = float(f)    # Transtypage
>>> type(g)
float
```

Entrées de base

On rappelle que Python est un langage *dynamique* (les variables peuvent changer de type au gré des affectations) mais néanmoins *fortement typé* (contrôle de la cohérence des types) :

```
>>> i = input("Entrez un entier : ")
Entrez un entier : 3
>>> i
'3'
>>> iplus = int(input("Entrez un entier : ")) + 1
Entrez un entier : 3
>>> iplus
4
>>> ibug = input("Entrez un entier : ") + 1
Entrez un entier : 3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: must be str, not int
```

Sortie de base

La fonction `print()` reste indispensable aux affichages dans les scripts. Elle se charge d'afficher la représentation textuelle des informations en paramètre, en plaçant un blanc séparateur entre deux informations, et en faisant un retour à la ligne à la fin de l'affichage. (le séparateur et la fin de ligne peuvent être modifiés) :

```
>>> print('Hello World!')
Hello World!
>>> print()                                # Affiche une ligne blanche

>>> a, b = 2, 5
>>> print('Somme :', a + b, ';', a - b, 'est la différence et', a * b, 'le produit.')
Somme : 7 ; -3 est la différence et 10 le produit.
>>> print(a, b, sep="+++", end='@')        # Utilise un séparateur et une fin de ligne spécifiques
2+++5@
```

Exercices

- Écrire un programme qui, à partir de la saisie d'un rayon (r) et d'une hauteur (h), calcule le volume d'un cône droit en utilisant la formule :

$$V = \frac{\pi r^2 h}{3}$$

- Entrer un nombre de secondes et l'afficher sous le format : [J:HH:MM:SS]. Chaque lettre J (jour), H (heure), M (minute) et S (seconde) doit occuper exactement 1 digit (mettre des 0 pour compléter le format demandé).

Exemple d'exécution :

```
Nombre de secondes : 100000
```

```
Durée : [1:03:46:40]
```



- Chapitre 2 -

Structures de contrôle

Indentation significative

Guido VAN ROSSUM a conçu Python après avoir travaillé sur le langage de script « ABC ». Ce langage utilisait déjà le concept de l'indentation comme syntaxe pour délimiter des *blocs* d'instructions.

Exemple :

```
if ph == 7:  
    print("Le potentiel hydrogène (pH) est exactement 7.")  
    print("La solution est neutre.")
```

Instructions composées

Une **instruction composée** se compose :

- ▣ d'une **ligne d'introduction** terminée par le caractère « deux-points » (:)
 - ▣ suivie d'un **bloc d'instructions indenté**. On utilise par convention quatre espaces par indentation et on n'utilise pas les tabulations mais
- Exemple d'instruction composée simple :

```
ph = float(input("PH ? "))
if ph < 7:
    print("Le potentiel hydrogène (pH) est inférieur à 7.")
    print("C'est un acide.")
if ph > 7:
    print("Le potentiel hydrogène (pH) est supérieur à 7.")
    print("C'est une base.")
```

Instruction composée imbriquée

Toutes les instructions au même niveau d'indentation appartiennent au même bloc d'instructions, jusqu'à ce que l'indentation redevienne inférieure à ce niveau.

```
t = float(input("Température (°C) ? "))
print("Température 't' en degrés Celsius")
if t <= 0:
    print('Négative ou nulle : risque de gel')
else:
    print('Positive')
    if t > 25:
        print('Plus de 25 °C')
        print('Prévoir tee-shirt ou veste légère')
    elif t > 18 :
        print('Douce mais sans plus')
    else
        print('Positive mais <= 18 °C. Sortez couverts')
print("Évaluation terminée")
```

Choisir : if - [elif] - [else]

Afin d'aiguiller différemment le flux normal des instructions, on utilise des *instructions conditionnelles*, qui permettent de contrôler des alternatives d'exécution entre différents blocs d'instructions.

Exemple de contrôle d'une alternative :

```
>>> x = 5
>>> if x < 0:
...     print("x est négatif")
... elif x % 2 != 0:
...     print("x est positif et impair")
...     print ("ce qui est bien aussi !")
... else:
...     print("x n'est pas négatif et est pair")
... 
```

Choisir : if - [elif] - [else]

Il est possible d'enchaîner plusieurs blocs **elif** mais il n'y a qu'un seul bloc **else**.

Test d'une valeur booléenne :

```
>>> # Attention de bien choisir le nom de la variable booléenne !
>>> x = 8
>>> est_pair = (x % 2 == 0)
>>> est_pair
True
>>> if est_pair: # Mieux que "if est_pair == True:"
...     print('La condition est vraie')
...
La condition est vraie
```

Syntaxe compacte d'une alternative

- Pour trouver, par exemple, le minimum de deux nombres, on peut utiliser l'opérateur *ternaire* :

```
>>> x = 4
>>> y = 3
>>> if x < y:                                # Écriture classique
...     plus_petit = x
... else:
...     plus_petit = y
...
>>> print("Plus petit :", plus_petit)
Plus petit : 3
>>> plus_petit = x if x < y else y            # Utilisation de l'opérateur ternaire
>>> print("Plus petit :", plus_petit)
Plus petit : 3
>>> print(f"Plus petit : {x if x < y else y}") # Affichage formaté
Plus petit : 3
```

L' instructions match-case

L'instruction match-case, introduite dans **Python 3.10**, est comme une version superhero des structures if-elif-else. C'est l'équivalent du switch-case des autres langages de programmation.

Cette instruction de “Filtrage par motif structurel” permet de comparer une valeur à plusieurs modèles et d'exécuter du code en fonction du premier modèle correspondant.

L' instructions match-case

Syntaxe de Base

```
match sujet:  
  case modèle1:  
    # Code pour modèle1  
  case modèle2:  
    # Code pour modèle2  
  case _:  
    # Cas par défaut
```



Exemple

```
match jour:  
  case 1:  
    return 'Lundi'  
  case 2:  
    return 'Mardi'  
  case 3:  
    return 'Mercredi'  
  case 4:  
    return 'Jeudi'  
  case 5:  
    return 'Vendredi'  
  case 6:  
    return 'Samedi'  
  case 7:  
    return 'Dimanche'  
  case _:  
    return 'Pas un jour de la semaine'
```

L' instructions match-case

- Dans cet exemple, nous faisons correspondre la variable héros à différents noms de superhéros. Le cas `_` agit comme notre cas par défaut, attrapant tous les héros non reconnus.

```
match héros:  
    case "Spider-Man":  
        print("Avec de grands pouvoirs viennent de grandes responsabilités !")  
    case "Iron Man":  
        print("Je suis Iron Man.")  
    case "Black Widow":  
        print("Je suis toujours en train de ramasser après vous les garçons.")  
    case _:  
        print("Je ne connais pas ce superhéros, mais il est probablement incroyable !")
```

Cas combinés dans l'Instruction Match

Parfois, nous voulons gérer plusieurs modèles avec le même code. Nous utilisons l'opérateur `|` pour combiner les cas.

```
match nombre:
  case 0:
    print("Zéro est un nombre unique !")
  case 1 | 2 | 3:
    print("C'est un petit nombre positif.")
  case -1 | -2 | -3:
    print("C'est un petit nombre négatif.")
  case _:
    print("C'est un plus grand nombre.")
```

Liste en argument dans l'Instruction Match Case

L'instruction match-case n'est pas seulement pour des valeurs simples. Elle peut gérer des structures complexes comme des listes

```
match point:
    case [0, 0]:
        print("Origine")
    case [0, y]:
        print(f"Axe des y à y = {y}")
    case [x, 0]:
        print(f"Axe des x à x = {x}")
    case [x, y]:
        print(f"Point à ({x}, {y})")
    case _:
        print("Ce n'est pas un point 2D valide")
```

Utilisation de "if" dans la Clause "Case"

Pour ces moments où nous avons besoin de plus de précision dans notre correspondance, nous pouvons ajouter des conditions de garde utilisant `if` :

```
match age:
    case n if n < 0:
        print("Alerte voyageur dans le temps ! Age négatif détecté.")
    case n if n < 13:
        print("Vous êtes un enfant. Profitez de votre jeunesse !")
    case n if n < 20:
        print("Bienvenue dans les années teenage !")
    case n if n < 65:
        print("La vie adulte, c'est parti !")
    case _:
        print("La sagesse vient avec l'âge. Vous en avez plein le dos !")
```

Table de Référence Rapide

Fonctionnalité	Exemple
Match-Case de Base	<code>match valeur: case modèle: ...</code>
Cas par Défaut	<code>case _: ...</code>
Cas Combinés	<code>case modèle1 modèle2: ...</code>
Correspondance de Liste	<code>case [x, y]: ...</code>
Conditions de Garde	<code>case modèle if condition: ...</code>

Exercices

Exercice 1 : Utiliser les conditions if, elif et else pour vérifier la nature d'un nombre. Vérifier si ce nombre saisi par l'utilisateur est Positif, Négatif ou Nul.

Exercice 2 : Écrivez un programme pour entrer le numéro du jour de la semaine (1-7) et affichez le nom du jour de la semaine correspondant.

Jeux de devinette

Écrire un programme qui choisit aléatoirement un nombre dans un intervalle donné par l'utilisateur, puis demande de deviner ce nombre.

- Si l'utilisateur trouve le nombre mystère, le programme affiche gagné
- Si l'utilisateur ne trouve pas le nombre mystère, le programme affiche essayez encore. Et affiche une aide en encadrant le nombre mystère dans un intervalle plus restreint.

Pour aider le joueur à trouver la bonne réponse, on lui donne une indication qui nécessite de résoudre une équation de premier degré. On lui dit que s'il ajoute deux fois le nombre mystère à son nombre, il doit trouver un nombre calculé par le programme et qui est égal à $2 * n_{\text{Myst}} + n_{\text{Util}}$.

Instructions nécessaires : Vous avez besoin d'importer une bibliothèque qui permet de donner des nombres aléatoires. Cette bibliothèque s'appelle **random**. Et à partir de cette bibliothèque on importe la fonction **randint()**.

Notions de conteneur et d'itérable

- Un **conteneur** désigne un type de données permettant de stocker un ensemble d'autres données.
- Un **itérable** désigne un type de conteneur que l'on peut parcourir élément par élément ou qui est capable de fournir des séquences de valeurs à la demande.

Pour parcourir ces conteneurs, nous nous servons parfois de l'instruction `range()`, qui fournit un moyen commode pour générer une série de valeurs entières servant

```
>>> uneListe = list(range(6))  
>>> uneListe  
[0, 1, 2, 3, 4, 5]
```

La boucles FOR

La boucle **for** permet de parcourir un itérable permet d'accéder tour à tour à chaque valeur afin de la traiter dans le corps de la boucle :

```
>>> for lettre in "ciao":    # On peut itérer sur les caractères des chaînes
...     print(lettre, lettre.upper())
...
c C
i I
a A
o O

>>> for x in [2, 'a', 3.14]: # Notation pour une liste de valeurs (cf. chap. 4)
...     # Le typage dynamique de Python permet à x de recevoir à
...     # chaque itération une valeur d'un type différent
...     print(x, x*2)
...
2 4
a aa
3.14 6.28
```

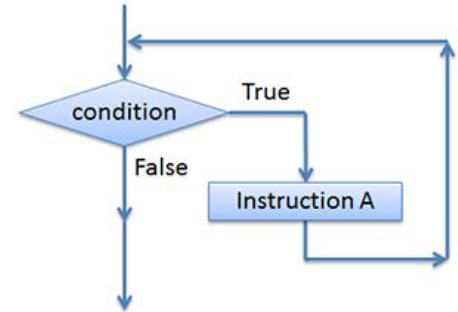
La boucles FOR

```
>>> for i in range(6):           # Générateur de séquences d'entiers à la demande
...     print(i, i ** 2)
...
0 0
1 1
2 4
3 9
4 16
5 25
>>> nb_voyelles = 0
>>> for lettre in "Python est un langage fort sympa":
...     if lettre.lower() in "aeiouy":
...         nb_voyelles = nb_voyelles + 1
...
>>> nb_voyelles
10
```

Répéter sous condition : while

- Répéter une portion de code tant qu'une expression booléenne est vraie (chaque répétition est appelée une *itération*) :

```
>>> x, cpt = 257, 0
>>> sauve = x
>>> while x > 1:
...     x = x // 2      # Division avec troncature
...     cpt = cpt + 1   # Incrémentation
...
>>> print("Approximation de log2 de", sauve, ":", cpt)
Approximation de log2 de 257 : 8
```



Répéter sous condition : while

- Utilisation classique, la *saisie filtrée* d'une valeur numérique (ne pas oublier d'*adapter le type* de la saisie, car on se rappelle que `input()` retourne une *chaîne* correspondant à la saisie de l'utilisateur) :

```
n = int(input('Entrez un entier [1 .. 10] : '))  
while not(1 <= n <= 10):  
    n = int(input('Entrez un entier [1 .. 10], S.V.P. : '))
```

Répéter sous condition : while

- Si la saisie est compliquée, on peut utiliser une variable « drapeau » (c'est-à-dire booléenne) :

```
saisie_ok = False
while not saisie_ok:    # Tant que saisie_ok est False, faire :
    a = int(input("Entrez un nombre entier de 1 à 100 divisible par 3 : "))
    b = int(input("Entrez un nombre entier pair supérieur à " + str(a) + ": "))
    saisie_ok = (1 <= a <= 100) and (a % 3 == 0) and (b % 2 == 0) and (a < b)
print("Ok")
```

Interrompre une boucle : break

- Sort immédiatement du corps de la boucle **for** ou **while** en cours contenant le **break** et passe à l'instruction suivante après la boucle :

```
for x in range(1, 11):  
    if x == 5:  
        break  
    print(x, end=" ") # end=" " remplace le retour à la ligne du print() par un simple espace  
  
print("Boucle interrompue pour x =", x)
```

- Ce qui produit :

```
1 2 3 4 Boucle interrompue pour x = 5
```

Équivalent du do...while

Dans de nombreux langages, il existe une instruction **do...while** qui permet de créer une boucle pour laquelle on ne connaît pas à l'avance le nombre de répétition, mais qui doit s'exécuter au moins une fois.

Cette instruction n'existe pas en Python, mais on peut facilement reproduire son fonctionnement de la façon suivante :

```
while True:
    n = int(input("donnez un entier > 0 : "))
    print("vous avez fourni", n)
    if n > 0:
        break
    print("reponse correcte")
```

La clause else dans une boucle

Signalons une syntaxe des boucles spécifique à Python. Les boucles **while** et **for** peuvent posséder une clause **else** qui ne s'exécute que si la boucle se termine sans interruption par **break**.

```
for n in range(2, 8):  
    for x in range(2, n):  
        if n % x == 0:  
            print(n, "egale", x, "*", n/x)  
            break  
    else:  
        print(n, "est un nombre premier")
```



```
2 est un nombre premier  
3 est un nombre premier  
4 egale 2 * 2.0  
5 est un nombre premier  
6 egale 2 * 3.0  
7 est un nombre premier
```

Court-circuiter une boucle : continue

Passe immédiatement à l'itération suivante de la boucle **for** ou **while** en cours contenant l'instruction; reprend à la ligne d'introduction de la boucle pour préparer l'itération suivante.

Ceci permet d'ignorer des valeurs ou des cas, selon des critères choisis, lors de traitements répétitifs :

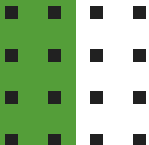
```
for x in range(1, 11):  
    if x == 5:  
        continue  
    print(x, end=" ")
```

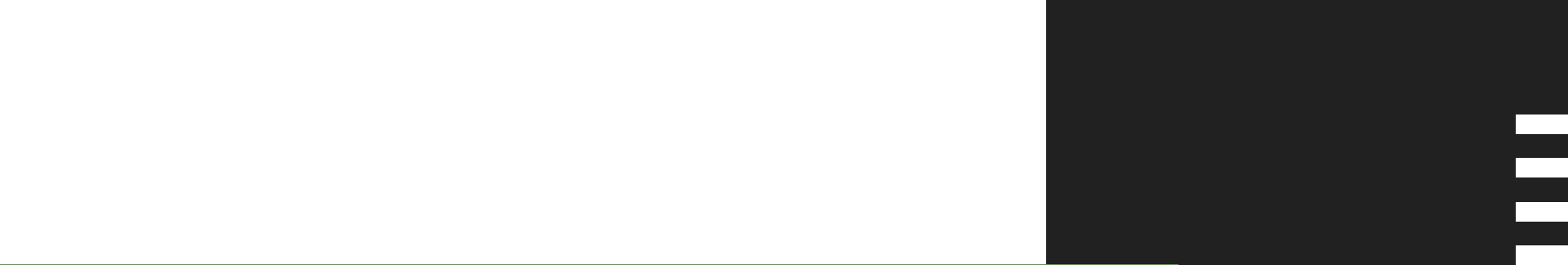
La boucle a sauté la valeur 5 :

1 2 3 4 6 7 8 9 10

Exercice

Proposez un exemple d'application des boucles **while** et **for** dans le cadre de votre projet





- Chapitre 3 -

Conteneurs standard



Les Séquences

- De façon générale, un **conteneur** est un objet composite destiné à contenir d'autres objets.
- Une **séquence** est un conteneur *ordonné* d'éléments *indiqués par des entiers* indiquant leur position dans le conteneur.
- Python dispose de trois types prédéfinis de séquences utilisés couramment : les **chaînes** (vues précédemment), les **listes** et les **tuples** .
- Les séquences sont numérotées à partir de 0.

Les opérations sur les objets de type séquentiel

Les types prédéfinis de séquences Python (chaîne, liste et tuple) ont en commun les opérations résumées dans le tableau suivant :

- **s** et **t** désignent deux séquences du même type,
- **x** un élément de la séquence
- **i, j** et **k** des entiers :

Opération	Signification
<code>x in s</code>	True si <code>s</code> contient <code>x</code> , False sinon
<code>x not in s</code>	True si <code>s</code> ne contient pas <code>x</code> , False sinon
<code>s + t</code>	concaténation de <code>s</code> et <code>t</code>
<code>s * n</code> ou <code>n * s</code>	<code>n</code> copies (superficielles) concaténées de <code>s</code>
<code>s[i]</code>	<code>i</code> ^e élément de <code>s</code> (à partir de 0)
<code>s[i:j]</code>	tranche de <code>s</code> de <code>i</code> (inclus) à <code>j</code> (exclu)
<code>s[i:j:k]</code>	idem avec un pas de <code>k</code>
<code>len(s)</code>	nombre d'éléments de <code>s</code>
<code>max(s), min(s)</code>	plus grand, plus petit élément de <code>s</code>
<code>s.index(i)</code>	indice de la 1 ^{re} occurrence de <code>i</code> dans <code>s</code>
<code>s.count(i)</code>	nombre d'occurrences de <code>i</code> dans <code>s</code>

Les listes

- Une **liste** est une collection **ordonnée mutable** (modifiable) d'éléments éventuellement **hétérogènes**.
- Les éléments **séparés par des virgules**, et **entourés de crochets**.

Exemple simple de liste :

```
couleurs = ['trèfle', 'carreau', 'coeur', 'pique']
print(couleurs)           # ['trèfle', 'carreau', 'coeur', 'pique']
couleurs[1] = 14          # C'est le deuxième élément de la liste
print(couleurs)           # ['trèfle', 14, 'coeur', 'pique']
list1 = ['a', 'b']
list2 = [4, 2.718]
list3 = [list1, list2]    # Liste de listes
print(list3)              # [['a', 'b'], [4, 2.718]]
```

Initialisations de la liste

Construction d'une liste vide et d'une liste répétant n fois une séquence de base :

```
>>> truc = []                                # Autre syntaxe : truc = list()
>>> truc
[]
>>> machin = [0.0] * 3
>>> machin
[0.0, 0.0, 0.0]
```

Longueur de la liste et tests d'appartenance

Utilisation de l'itérateur d'entiers `range()`, longueur et test d'appartenance :

```
>>> liste_1 = list(range(4))    # range() : générateur de séquences d'entiers à la demande
>>> liste_1
[0, 1, 2, 3]
>>> liste_2 = list(range(4, 8))
>>> liste_2
[4, 5, 6, 7]
>>> liste_3 = list(range(2, 9, 2))
>>> liste_3
[2, 4, 6, 8]
>>> len(liste_1)
4
>>> 2 in liste_1, 8 in liste_2, 6 not in liste_3
(True, False, False)
```

Méthodes modificatrices

Voici quelques exemples de méthodes de modification des listes.

```
>>> nombres = [17, 38, 10, 25, 72]
>>> nombres.sort()           # Tri de la liste sur place
>>> nombres
[10, 17, 25, 38, 72]
>>> nombres.append(12)       # Ajout d'un élément à la fin
>>> nombres
[10, 17, 25, 38, 72, 12]

>>> nombres.reverse()        # Inversion des éléments de la liste
>>> nombres
[12, 72, 38, 25, 17, 10]
>>> nombres.remove(38)        # Suppression d'une valeur (autre syntaxe : del nombres[2])
>>> nombres
[12, 72, 25, 17, 10]
>>> nombres.extend([1, 2, 3]) # Ajout d'une séquence d'éléments à la fin
>>> nombres
[12, 72, 25, 17, 10, 1, 2, 3]
```

Méthodes de modification des listes

Syntaxe (listes)	Usage
<code>del lst[déb:fin[:pas]]</code>	► Suppression des items situés dans la tranche <code>déb</code> à <code>fin</code> de la liste <code>lst</code> . Les items situés après cette tranche sont tous décalés d'autant de crans de moins que nécessaire
<code>lst.pop()</code>	► Suppression et retour de la valeur du dernier item de la liste <code>lst</code>
<code>lst.pop(k)</code>	► Suppression et retour de la valeur de l'item d'index <code>k</code> de la liste <code>lst</code> . Les items situés après cet index sont tous décalés d'un cran de moins
<code>lst.remove(val)</code>	► Recherche du premier item de valeur <code>val</code> dans la liste, et suppression de cet item. Les items situés après celui trouvé sont tous décalés d'un cran de moins
<code>lst.clear()</code>	► Suppression de tous les items de la liste <code>lst</code> , qui devient donc vide. Autre syntaxe : <code>del lst[:]</code>

Méthodes de modification des listes

```
lst.sort()
```

► Tri des items de la liste `lst` par ordre croissant — les valeurs doivent être comparables. Argument optionnel `reversed=True` pour trier par ordre décroissant

```
lst.reverse()
```

► Inversion de l'ordre des items de la liste `lst`

```
lst.insert(k, val)
```

► Insère un item de valeur `val` à l'index `k` de la liste `lst`. Les items qui étaient situés à partir de cet index sont tous décalés d'un cran de plus

```
lst.append(val)
```

► Ajout d'un item de valeur `val` à la fin de la liste `lst`

```
lst.extend(séq)
```

► Ajout d'un ensemble de valeurs issues d'une séquence `séq` à la fin de la liste `lst`

Manipulation des index et des slices

La manipulation des index et des slices utilise la même syntaxe que celle déjà vue pour les chaînes :

```
>>> mots = ['jambon', 'sel', 'miel', 'confiture', 'beurre']
>>> mots[2:4] = []           # Effacement par affectation d'une liste vide
>>> mots
['jambon', 'sel', 'beurre']
>>> mots[1:3] = ['salade']
>>> mots
['jambon', 'salade']
>>> mots[1:] = ['mayonnaise', 'poulet', 'tomate']
>>> mots
['jambon', 'mayonnaise', 'poulet', 'tomate']
>>> mots[2:2] = ['miel']    # Insertion en 3e position
>>> mots
['jambon', 'mayonnaise', 'miel', 'poulet', 'tomate']
```

Les Tuples

- Un **tuple** est une collection **ordonnée immuable** (non modifiable) d'éléments éventuellement **hétérogènes**.
- Les éléments d'un tuple sont **séparés par des virgules**, et **optionnellement entourés de parenthèses**.
- Un tuple ne comportant qu'un seul élément (ou *singleton*) doit être obligatoirement noté avec une virgule terminale.

```
>>> mon_tuple = ('a', 2, [1, 3])      # Tuple de trois éléments
>>> ton_tuple = 'un', 'deux', 'trois' # Tuple de trois éléments
>>> s = (2.718,)                      # Singleton
>>> t = 'toto',                       # Singleton
```

Les Tuples

Construction d'un tuple vide :

```
>>> v = (,)          # Tuple vide  
>>> w = tuple()      # Tuple vide
```

- L'indexage des tuples s'utilise comme celui des listes et des chaînes
- Le parcours des tuples est plus rapide que celui des listes.
- On peut utiliser les **tuples pour définir des constantes**.

Les Tuples

Comme les chaînes de caractères, une fois construits, les tuples sont immutables !

```
>>> mon_tuple = (1, 2)
>>> mon_tuple[1] = 3          # Attention : on ne peut pas modifier un tuple
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

Séquences de séquences

Les séquences, comme du reste les autres conteneurs, peuvent être imbriquées. Par exemple :

```
>>> un_tuple = (1, 2, 3)
>>> sequences = [un_tuple, [4, 5]]
>>> for sequence in sequences:
...     for item in sequence:
...         print(item, end=' ')
...     print()
...
1 2 3
4 5
```

Séquences de séquences

Une liste imbriquée numérique est une représentation possible d'une matrice. La syntaxe d'accès à ses éléments nécessite un double indilage :

```
>>> matrix = [  
    [1, 2, 3, 4],  
    [5, 6, 7, 8],  
    [9, 10, 11, 12]]  
>>> for i in range(3):  
    for j in range(4):  
        print(f"{matrix[i][j]}", end=' ')
```

```
1 2 3 4 5 6 7 8 9 10 11 12
```

Exercice : Gestion des notes d'une classe

Chaque étudiant a une note sur 20. Écris un programme qui :

- Demande à l'utilisateur de saisir le nombre des étudiants ($1 < N < 36$).
- Demande à l'utilisateur de saisir les notes des étudiants.
- Les stocke dans une liste.
- Calcule la moyenne des notes de la classe.
- Affiche le nombre d'étudiants ayant une note supérieure à la moyenne.
- Affiche la note la plus élevée et la note la plus faible dans la classe.

Tables de hash

Les séquences sont des structures très souples mais possèdent des limitations.

En effet :

- le test d'appartenance d'un élément à une séquence est de complexité linéaire ;
- on aimerait disposer d'une notation d'accès *associative* et pas seulement *indicée*.

Une **table de hash** est constituée d'un tableau et d'une fonction particulière. Le tableau possède un certain nombre de cases et la fonction dite « de hash » a pour rôle de créer une correspondance entre un objet X et un entier. Ainsi la fonction de hash calcule très rapidement le numéro de la case du tableau où est stocké l'objet sous la forme d'un couple (clé, valeur).

Tables de hash

- Une **table de hash** est un conteneur non ordonné d'éléments indexés par des clés, avec un accès très rapide à un élément à partir de sa clé, chaque clé ne pouvant être présente qu'une seule fois dans la collection.
- Dans une table de hash, la gestion du stockage (insertion, recherche, remplacement, etc.) est indépendante du nombre d'éléments de la table, sa complexité ne dépend que de la vitesse de la fonction de hash.

Tables de hash

on veut stocker l'association entre la clé ('Joe') et sa valeur (37) dans le tableau :

- on passe la clé à la fonction de hash qui retourne l'entier 1,
- le couple ('Joe', 37) est donc stocké dans le case n° 1 du tableau ;
- de même, le couple ('Bob', 53) est stocké dans le case n° 4 du tableau.

Si maintenant on désire accéder (par exemple pour l'afficher) à la valeur de la clé 'Bob', la fonction .), retourne 53.

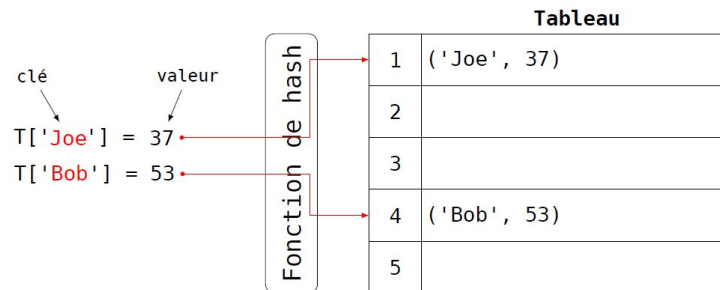


FIGURE 4.5 – Principe d'une table de hash

Dictionnaires

Les dictionnaires constituent un type composite qui n'appartient pas aux séquences car ils n'en partagent pas les caractéristiques communes. À la différence des séquences, qui sont indexées par des nombres, les dictionnaires sont indexés par des clés.

Un dictionnaire est une table de hash mutable

- Il permet de stocker des couples (ou paires) (clé, valeur) avec des valeurs de tout type, éventuellement hétérogènes, les clés ayant comme contrainte d'être *hashables* (Dont les valeurs permettent de calculer une valeur entière – la valeur de *hash* qui ne change pas au cours du temps).

Dictionnaires

En Python, les dictionnaires sont implémentés très efficacement grâce à un algorithme sophistiqué des fonctions de hash. Il permet un accès très rapide à partir de la clé *via* un index dans une « table de hachage ».

Syntaxe :

Couples notés clé : valeur, séparés les uns des autres par des virgules et entourés d'accolades.

`{"clé0" : valeur0 , "clé1" : valeur1 , ... }`

Dictionnaires

Exemples de création

```
>>> d1 = {}                                # Dictionnaire vide. Autre notation : d1 = dict()
>>> d1["nom"] = 3                          # La clé "nom" reçoit la valeur 3
>>> d1["taille"] = 176
>>> d1
{'nom': 3, 'taille': 176}
>>> d2 = {"nom": 3, "taille": 176}          # Définition en extension des couples (clé:valeur)
>>> d2
{'nom': 3, 'taille': 176}
>>> d3 = dict(nom=3, taille=176)           # Utilisation de paramètres nommés
                                           # (syntaxe d'appel de fonction)
>>> d3
{'taille': 176, 'nom': 3}
>>> d4 = dict([("nom", 3), ("taille", 176)]) # Utilisation d'une liste de couples clés/valeurs
>>> d4
{'nom': 3, 'taille': 176}
```

Méthodes applicables aux dictionnaires

```
>>> tel = {'jack': 4098, 'sape': 4139, 'guido': 4127}
>>> tel
{'guido': 4127, 'jack': 4098, 'sape': 4139}  # Un dictionnaire n'est pas ordonné
>>> tel['jack']                               # Valeur de la clé 'jack'
4098
>>> del tel['sape']                           # Suppression d'un couple (clé : valeur)
>>> tel.keys()                               # Clés de tel
dict_keys(['jack', 'guido'])
>>> tel.values()                             # Valeurs de tel
dict_values([4098, 4127])
>>> 'guido' in tel, 'jack' not in tel         # Teste l'appartenance d'une clé au dictionnaire
(True, False)
```

Les vues

En plus de ces opérations, les dictionnaires possèdent les méthodes **keys**, **values** et **items** qui retournent une *vue*.

Une vue est un objet itérable, mis à jour en même temps que le dictionnaire :

```
>>> sac = {3:'pommes', 8:'yaourts', 4:'jambon', 1:'pain'}
>>> sac.items()
dict_items([(3, 'pommes'), (8, 'yaourts'), (4, 'jambon'), (1, 'pain')])
>>> sac.values()
dict_values(['pommes', 'yaourts', 'jambon', 'pain'])
>>> k = sac.keys()           # k est une vue
>>> k
dict_keys([3, 8, 4, 1])
>>> del sac[8]               # Suppression d'un couple
>>> sac[7] = 'beurre salé'   # Ajout d'un couple
>>> k                        # k est automatiquement mis à jour
dict_keys([3, 4, 1, 7])
>>> 8 in k, 7 in k          # Les vues supportent les tests d'appartenance
(False, True)
```

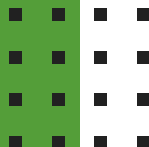
Ensembles (set)

Les ensembles en Python forment le type **set**. Ce sont également des tables de hash, comme les dictionnaires, mais ils ne stockent que des clés.

Un ensemble est une collection itérable non ordonnée d'éléments *hashables* uniques.

Syntaxe :

Valeurs séparées les unes des autres par des virgules et entourées d'accolades.



Ensembles (set)

Un **set** est une transposition informatique de la notion d'ensemble mathématique.

En Python, il existe deux types d'ensembles : les ensembles mutables (**set**) et les ensembles immutables (**frozenset**).

On retrouve ici les mêmes différences qu'entre les listes et les tuples.

```
>>> couleurs = {'trefle', 'carreau', 'coeur', 'pique'} # Expression littérale
>>> chiffres = set(range(10)) # Construction à partir des éléments d'un itérable
>>> couleurs
{'coeur', 'trefle', 'pique', 'carreau'}
>>> chiffres
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
```

Opérations sur les ensembles

Exemples d'opérations sur les ensembles

```
X = set('abcdef')    # X = {'a', 'b', 'c', 'd', 'e', 'f'}
Y = set('efghf')     # Y = {'e', 'f', 'g', 'h'} pas de duplication : qu'un seul 'f'
'b' in X, 'c' in Y   # (True, False)
X - Y # {'a', 'b', 'c', 'd'} ensemble des éléments de X qui ne sont pas dans Y
Y - X # {'g', 'h'} ensemble des éléments de Y qui ne sont pas dans X
X | Y # {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'} union
X & Y # {'e', 'f'} intersection
X ^ Y # {'a', 'b', 'c', 'd', 'g', 'h'} ensemble des éléments qui sont soit dans X, soit dans Y
```

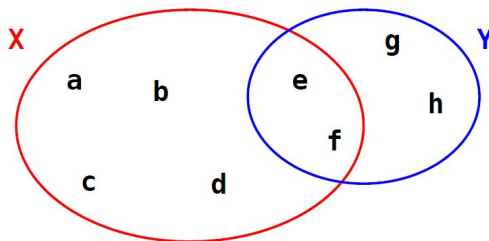


FIGURE 4.7 – Opérations sur les ensembles

Remarque : cas d'application importante

- Dans une séquence de taille N , il faut parcourir en moyenne $N/2$ éléments pour trouver un nombre présent, et N éléments pour vérifier qu'un nombre est absent.
- Dans un ensemble (**set**), quelle que soit la taille, une fois la clé de hachage calculée (c'est quasi immédiat pour les nombres entiers, et la valeur calculée est conservée par exemple pour les chaînes), la recherche de présence/absence d'un nombre est immédiate.

Itérer sur les conteneurs

Les techniques suivantes sont classiques et très utiles.

Obtenir clés et valeurs en bouclant sur un dictionnaire

```
knight = {"Gallahad": "the pure", "Robin": "the brave"}  
for k, v in knight.items():  
    print(k, v)  
# Gallahad the pure  
# Robin the brave
```

Obtenir indice et élément en bouclant sur une liste

```
>>> for i, v in enumerate(["tic", "tac", "toe"]):  
...     print(i, '->', v)  
...  
0 -> tic  
1 -> tac  
2 -> toe
```

Boucler sur deux séquences (fonction ZIP)

La fonction `zip()` fait allusion à la fermeture Éclair, qui joint et entrelace deux rangées de dents. Elle permet de fournir à chaque itération les valeurs de même index issues de plusieurs séquences.

```
>>> questions = ['name', 'quest', 'favorite color']
>>> answers = ['Lancelot', 'the Holy Grail', 'blue']
>>> for question, answer in zip(questions, answers):
...     print('What is your', question, '? It is', answer)
...
What is your name ? It is Lancelot
What is your quest ? It is the Holy Grail
What is your favorite color ? It is blue
```

Itérer sur les séquences

Obtenir une séquence inversée (la séquence initiale est inchangée)

```
for i in reversed(range(1, 10, 2)):
    print(i, end=" ")      # 9 7 5 3 1
```

Obtenir une séquence triée (la séquence initiale est inchangée)

```
basket = ["apple", "orange", "apple", "pear", "orange", "banana"]
for f in sorted(basket):
    print(f, end=" ")      # apple apple banana orange pear
```

Obtenir une séquence triée à éléments uniques (la séquence initiale est inchangée)

```
basket = ["apple", "orange", "apple", "pear", "orange", "banana"]
for f in sorted(set(basket)):
    print(f, end=" ")      # apple banana orange pear
```

Exercices

Une entreprise vend différents produits en ligne. On vous demande de traiter un fichier ou une liste d'achats effectués par les clients pendant un mois. Chaque achat contient un identifiant client, un produit acheté et la quantité de ce produit.

```
achats = [  
    {"client_id": 101, "produit": "pomme", "quantite": 3},  
    {"client_id": 102, "produit": "banane", "quantite": 2},  
    {"client_id": 103, "produit": "orange", "quantite": 5},  
    {"client_id": 101, "produit": "kiwi", "quantite": 1},  
    {"client_id": 104, "produit": "pomme", "quantite": 2},  
    {"client_id": 105, "produit": "melon", "quantite": 6},  
    {"client_id": 102, "produit": "melon", "quantite": 3},  
    {"client_id": 101, "produit": "banane", "quantite": 1},  
    {"client_id": 106, "produit": "orange", "quantite": 4}  
]
```

Tâches :

- Analyser le nombre total de ventes par produit. Utilise un dictionnaire où chaque produit est une clé et la valeur associée est le total des quantités vendues.
- Trouver les id clients uniques ayant effectué un achat. Utilise un set pour éviter les doublons.
- Lister les clients ayant acheté plus de 2 commandes. Utilise une liste pour stocker ces clients.



- Chapitre 4 -

Les fonctions et espaces de nommage

Définition et syntaxe

Une **fonction** est un ensemble d'instructions regroupées sous un *nom* et s'exécutant à la demande (l'*appel* de la fonction).

La **définition** d'une fonction se compose :

- du **mot-clé** **def** suivi de l'identificateur de la fonction, de parenthèses entourant les paramètres de la fonction séparée par des virgules, et du caractère « deux-points » qui termine toujours une instruction composée (c'est l'*en-tête* de la fonction) ;
- d'une chaîne de documentation indentée comme le corps de la fonction ;
- du bloc d'instructions indenté par rapport à la ligne de définition, et qui constitue le corps de la fonction.

Définition et syntaxe

- Le bloc d'instructions est *obligatoire*. S'il est vide, on emploie l'instruction **pass**.
- La documentation, bien que facultative, est *fortement* conseillée

```
def volume_ellipsoide(a, b, c):  
    """Retourne le volume d'un ellipsoïde de demi-grands axes a, b et c."""  
    return 3.14 * a * b * c * 4 / 3  
  
def fonction_vide():  
    """Une fonction sans corps doit contenir l'instruction 'pass'."""  
    pass
```

Passage des arguments

En Python, les arguments sont passés *par affectation* : chaque argument de l'appel référence, *dans l'ordre*, un paramètre de la définition de la fonction.

Exemple sans l'instruction **return**, ce qu'on appelle souvent une « procédure ». Dans ce cas, la fonction renvoie implicitement la valeur **None**

```
:
```

```
def table(base, debut, fin):  
    """Affiche la table de multiplication des <base> de <debut> à <fin>."""  
    n = debut  
    while n <= fin:  
        print(n, 'x', base, '=', n * base)  
        n += 1  
  
# Exemple d'appel  
table(7, 2, 8)  
# 2 x 7 = 14 3 x 7 = 21 4 x 7 = 28 5 x 7 = 35 6 x 7 = 42 7 x 7 = 49 8 x 7 = 56  
  
# Autre exemple du même appel, mais en nommant les paramètres dont on peut alors changer l'ordre  
table(debut=2, fin=8, base=7)
```

Type de retour unique

Avec utilisation d'un **return** d'une valeur unique. Le résultat de l'évaluation de la fonction peut être utilisé dans une expression ou affecté à une variable :

```
from math import pi

def cube(x):
    """Retourne le cube de l'argument."""
    return x**3

def volume_sphere(r):
    """Retourne le volume d'une sphère de rayon <r>;"""
    return 4.0 * pi * cube(r) / 3.0

# Saisie du rayon et affichage du volume de la sphère
rayon = float(input('Rayon : '))
print("Volume de la sphère =", volumeSphere(rayon))
```

Fonction à plusieurs retours

Avec utilisation d'un **return** de multiples valeurs. Les résultats sont retournés par Python dans un tuple. Celui-ci peut être « décapsulé » dans un nombre correspondant de variables, ou bien stocké dans une seule variable en tant que tuple (dont les éléments pourront ultérieurement être accédés par leur indice) :

```
import math

def surfaceVolumeSphere(r):
    surf = 4.0 * math.pi * r**2
    vol = surf * r/3
    return surf, vol

# Programme principal ~~~~~
rayon = float(input('Rayon : '))
s, v = surfaceVolumeSphere(rayon)
print("Sphère de surface {:g} et de volume {:g}".format(s, v))
```

Passage d'une fonction en paramètre

En Python, **une fonction est un objet manipulable** comme toute valeur, ce qui signifie entre autres qu'une **variable peut référencer une fonction**.

On peut donc transmettre une fonction comme paramètre et on peut retourner une fonction :

```
>>> def double_filtre(lst, fct_filtre):
...     lres = []
...     for v in lst:
...         if fct_filtre(v):
...             lres.append(v * 2)
...     return lres
...
>>> def fgrand(n):
...     return n>10
...
>>> double_filtre(range(1, 20, 3), fgrand)
[26, 32, 38]
>>> def fpair(n):
...     return n%2 == 0
...
>>> double_filtre(range(1, 20, 3), fpair)
[8, 20, 32]
```

Appel avec des arguments nommés

À l'appel d'une fonction, on peut utiliser des arguments *nommés* (Il est souvent plus aisé de se souvenir du nom des paramètres plutôt que de leur ordre). Dans ce cas, l'ordre d'appel est libre.

```
def table(base, debut, fin):  
    """Affiche la table de multiplication des <base> de <debut> à <fin>."""  
    n = debut  
    while n <= fin:  
        print(n, 'x', base, '=', n * base)  
        n += 1  
table(debut=2, fin=8, base=7)
```

Paramètres avec valeur par défaut

- Il est possible de spécifier, lors de la déclaration, des valeurs par défaut à utiliser pour les arguments. Cela permet, lors de l'appel, de ne pas avoir à spécifier les paramètres correspondants.
- Il est également possible, en combinant les valeurs par défaut et le nommage des paramètres, de n'indiquer à l'appel que les paramètres dont on désire modifier la valeur de l'argument.
- Il est par contre nécessaire, lors de la définition, de regrouper tous les paramètres optionnels avec leurs valeurs par défaut à la fin de la liste des paramètres.

Paramètres avec valeur par défaut

Exemple :

```
>>> def accueil(nom, prenom, depart="MP", semestre="S2"):
...     print(prenom, nom, "Département", depart, "semestre", semestre)
...
>>> accueil("Deuf", "John")
John Deuf Département MP semestre S2
>>> accueil("Paradise", "Eve", "Info")
Eve Paradise Département Info semestre S2
>>> accueil("Annie", "Steph", semestre="S3")
Steph Annie Département MP semestre S3
```

Paramètres avec valeur par défaut

- On utilise de préférence des valeurs par défaut *immuables* (types **int**, **float**, **str**, **bool**, **tuple**...) car la modification d'un paramètre par un premier appel est visible les fois suivantes.
- Si on a besoin d'une valeur par défaut qui soit *mutable* (**list**, **dict**), on utilise la valeur prédéfinie **None** et on fait un test dans la fonction pour mettre en place la valeur par défaut :

```
def maFonction(liste=None):  
    if liste is None:  
        liste = [1, 3]
```

Nombre d'arguments arbitraire sous forme de tuple

Le passage d'un nombre arbitraire d'arguments est permis en utilisant la notation d'un paramètre final `*<nom_parametre>`.

Les arguments surnuméraires sont alors transmis sous la forme d'un **tuple** affecté à ce paramètre

```
def f(*args):  
    print(args)
```

```
# Exemples d'appel :
```

```
f()           # ()
```

```
f(1)          # (1,)
```

```
f(1, 2, 3, 4) # (1, 2, 3, 4)
```

Nombre d'arguments arbitraire sous forme de tuple

Réciproquement, il est aussi possible de passer un tuple (en fait une séquence) à l'appel, qui sera *décapsulé* en une liste de paramètres ordonnés.

```
def somme(a, b, c):  
    return a+b+c  
  
# Exemple d'appel :  
elements = (2, 4, 6)  
print(somme(*elements))      # 12
```

Nombre d'arguments arbitraire : les dictionnaires

De la même façon, il est possible d'autoriser le passage d'un nombre arbitraire d'arguments nommés en plus de ceux prévus lors de la définition en utilisant la notation d'un paramètre final `**<nom_parametre>`.

Les arguments surnuméraires nommés sont alors transmis sous la forme d'un dictionnaire affecté à ce paramètre (que l'on appelle généralement **kwargs** pour *keyword args*).

Nombre d'arguments arbitraire : les dictionnaires

Réciproquement, il est aussi possible de passer un dictionnaire à l'appel d'une fonction, qui sera décapsulé, chaque clé étant liée au paramètre correspondant de la fonction.

```
def unDict(**kwargs):  
    return kwargs  
  
# Exemples d'appels  
## par des paramètres nommés :  
print(unDict(a=23, b=42))      # {'a': 23, 'b': 42}  
  
## en fournissant un dictionnaire :  
mots = {'d': 85, 'e': 14, 'f': 9}  
print(unDict(**mots))          # {'e': 14, 'd': 85, 'f': 9}
```

Argument mutable/immutable

Lorsque l'on passe à une fonction un **argument immutable** (entier, chaîne...), il peut être utilisé sans restriction et sans avoir à se poser de question.

Par contre, lorsque l'on passe à une fonction un **argument mutable** (liste, dictionnaire...), alors il faut avoir conscience que toute modification sur celui-ci dans la fonction persistera après la sortie de la fonction, on appelle cela un « *effet de bord* » car la fonction modifie des données qui sont définies hors de sa portée locale (on appelle aussi « effet de bord » le fait de modifier une variable globale).

Argument immutable

Opération avec paramètre immutable

```
def additionne_1(x):  
    x = x + 1  
    return x
```

```
a = 3
```

```
print(additionne_1(a)) # 4
```

```
print(a) # 3
```

(a n'est pas modifié)

Argument mutable

```
# Opération avec paramètre mutable
```

```
def ajoute_1(x):  
    x.append(1)  
    return x
```

```
lst = [1, 4, 5]
```

```
print(ajoute_1(lst))    # [1, 4, 5, 1]  
print(lst)              # [1, 4, 5, 1]    (lst est modifié à chaque appel)  
print(ajoute_1(lst))    # [1, 4, 5, 1, 1]  
print(lst)              # [1, 4, 5, 1, 1]  
print(ajoute_1(lst))    # [1, 4, 5, 1, 1, 1]  
print(lst)              # [1, 4, 5, 1, 1, 1]
```

Argument mutable

Cet effet de bord, s'il est bien maîtrisé, possède un avantage. Comme il n'y a pas de création d'objet, le passage est économique en encombrement mémoire.

```
>>> def ajoute_2(x):  
    x = x[:]      # Copie de l'objet mutable  
    x.append(2)  
    return x  
  
>>> m = [7, 4, 9]  
>>> ajoute_2(m)  
[7, 4, 9, 2]  
>>> m  
[7, 4, 9]      # m n'a pas subi d'effet de bord : la liste est inchangée
```

Fonction anonyme

Issue de langages fonctionnels (comme OCaml, Haskell, Lisp), la directive **lambda** permet de définir une *fonction anonyme* comportant un bloc d'instructions limité à une expression dont l'évaluation fournit la valeur de retour de la fonction.

Syntaxe



lambda [paramètres]: expression

Fonction anonyme

Ces fonctions anonymes sont souvent utilisées lorsqu'il s'agit simplement d'adapter l'appel à une fonction existante

```
>>> # Retourne 's' si son argument est différent de 1, une chaîne vide sinon
>>> s = lambda x: "" if x == 1 else "s"
>>> s(1), s(3)
(' ', 's')
>>> # On peut utiliser la fonction print() en tant qu'expression
>>> majorite = lambda x : print('mineur') if x < 18 else print('majeur')
>>> majorite(15)
mineur
>>> majorite(25)
majeur
```

Fonction anonyme

```
>>> # Retourne le tuple somme et différence de ses deux arguments
>>> t = lambda x, y: (x+y, x-y)
>>> t(5, 2)
(7, 3)
```

Associées aux fermetures, les fonctions anonymes permettent de créer simplement des fonctions de calcul paramétrées :

```
def polynome(a, b, c) :
    return lambda x : a*x**2 + b*x + c

p1 = polynome(3, -1, 4)
p2 = polynome(-1, 2, 0)
print(p1(1))    # 6
print(p1(2))    # 14
print(p2(10))   # -80
```

Exercices

Proposez un exemple d'application des fonctions dans le cadre de votre projet

